

National School of Cyber Security

Foundation Training Department

Linux System Audit and Monitoring using Shell Scripting Report

Realized by : MAHIOUT Meriem & AMIRI Maria.

Section : A, Group : A3 & A2.

Date : Monday, March 30rd, 2026.

Teacher : Dr. BENTRAD Sassi.

Course: INTRODUCTION TO OPERATING SYSTEMS 2.

1. Introduction

In the field of cybersecurity, system auditing is essential for maintaining the integrity, availability, and security of information systems. Administrators must continuously monitor both hardware and software configurations to detect vulnerabilities, unauthorized changes, and abnormal system behavior.

This project introduces an automated **Bash-based auditing system** for Linux environments, designed to replace inefficient manual monitoring. By programmatically collecting hardware and software data, the tool detects vulnerabilities and unauthorized changes to ensure system integrity. It features scalable automation and generates structured short and full reports for streamlined security administration.

2. Project Objectives

The objectives of this project are:

- ✓ To design an automated audit system using shell scripting.
- ✓ To collect detailed hardware and software information.
- ✓ To generate structured and readable reports (short and full).
- ✓ To automate execution using cron jobs.
- ✓ To implement email-based report delivery.
- ✓ To introduce basic remote monitoring capabilities.
- ✓ To apply cybersecurity principles in system auditing.

3. System Architecture and Design

The system follows a **modular architecture**, where each component is implemented as an independent function. This approach improves code readability, maintainability, and scalability.

3.1 System Components

The system is divided into the following main modules:

A. Hardware Audit Modules

- ◆ CPU Information

- ◆ Memory (RAM)
- ◆ Disk and Storage
- ◆ Network Interfaces (MAC & IP addresses)

- ◆ GPU (if available)
- ◆ Motherboard Information
- ◆ USB Devices

B. Software Audit Modules

- ◆ Operating System Information.
- ◆ Installed Packages.
- ◆ Logged-in Users.

- ◆ Running Services.
- ◆ Active Processes.
- ◆ Open Ports.

C. Reporting Module

- ◆ Short Report (summary view).
- ◆ Full Report (detailed view).

D. Automation and Monitoring

- ◆ Cron-based scheduling.
- ◆ Email report transmission.
- ◆ Logging system.

4. Implementation Details

4.1 Hardware Audit Implementation

The hardware audit module collects system-level information using built-in Linux utilities and system files.

- CPU, memory, and disk information are obtained using standard Linux commands.
- Network interfaces are analyzed to extract MAC and IP addresses.
- Motherboard information is retrieved from system files located in `/sys/devices/virtual/dmi/id/`.
- USB devices are detected through `/sys/bus/usb/devices/`.

To ensure compatibility in restricted environments, the implementation prioritizes **native Linux virtual file systems** (`/sys`, `/proc`) instead of relying on external tools.

4.2 Software Audit Implementation

The software audit module focuses on system activity and runtime information:

- Logged-in users are identified using session tracking commands.
- Running services are retrieved via the system service manager.
- Active processes are analyzed and filtered based on CPU usage.

- Open ports are identified to monitor network exposure.

Filtering techniques are applied to ensure that only relevant and useful information is included in the reports.

4.3 Report Generation

The system generates two types of reports:

4.3.1 Short Report

Provides a quick overview of the system

Includes summarized information such as:

- Primary IP and MAC address.
- Basic motherboard details.
- Number of USB devices.
- Counts of services, processes, and open ports.

4.3.2 Full Report

Provides a detailed system audit

Includes:

- All network interfaces.
- Complete motherboard information.
- Detailed USB device list.
- Logged-in users with session details.
- Running services.
- Top active processes.
- Open ports.

4.3.3 Formatting Strategy

- Structured sections with headers.
- Aligned tabular output.
- Clear separation between modules.
- Inclusion of metadata (date, hostname, user).

5. Automation Using Cron

Automation is implemented using cron jobs to schedule periodic execution of the audit scripts.

This configuration executes the audit system daily at 04:00 PM.

6. Email Transmission

The system supports automatic report delivery via email.

6.1.Implementation

- The script detects available tools such as **msmtp**.
- The generated report is sent to a configured email address.
- The subject includes system identification information.

6.2.Limitations

Email functionality may require prior SMTP configuration depending on the system environment.

7. Remote Monitoring Concept

Although full remote monitoring implementation may vary, the system is designed to support:

- Remote execution via SSH.
- Centralized collection of audit reports.
- Periodic monitoring of multiple systems.

This aligns with cybersecurity practices where systems are monitored remotely for threats and anomalies.

8. Security Considerations

Several security measures were considered:

- ✓ Use of read-only system files (`/sys`, `/proc`).
- ✓ Avoidance of unnecessary privileged operations.
- ✓ Limitation of sensitive data exposure.
- ✓ Use of logging for audit traceability.
- ✓ Secure handling of automated tasks (cron jobs).

9. Challenges Encountered

During the development of this project, several challenges were encountered:

- Limited permissions for accessing certain hardware information.
- Absence of some system tools in restricted environments.
- Formatting raw command output into readable reports.
- Ensuring compatibility across different Linux configurations.
- Debugging cron job execution issues.

These challenges were addressed through adaptive scripting techniques and the use of native system resources.

10. Conclusion

This project demonstrates the effectiveness of shell scripting in automating system auditing tasks. The developed solution provides a structured and efficient way to collect and analyze both hardware and software information.

By integrating reporting, automation, and monitoring features, the system aligns with real-world cybersecurity practices and enhances system visibility.

Future improvements may include:

- Web-based report generation.
- Advanced alerting mechanisms.
- Integration with centralized monitoring systems.

11. References

- Linux manual pages (man).
- Bash scripting documentation.
- Ubuntu system documentation.
- Virtual file systems: `/proc`, `/sys`.